

Class07: Machine Learning 1

Joli DeLucia (A17320476)

Background

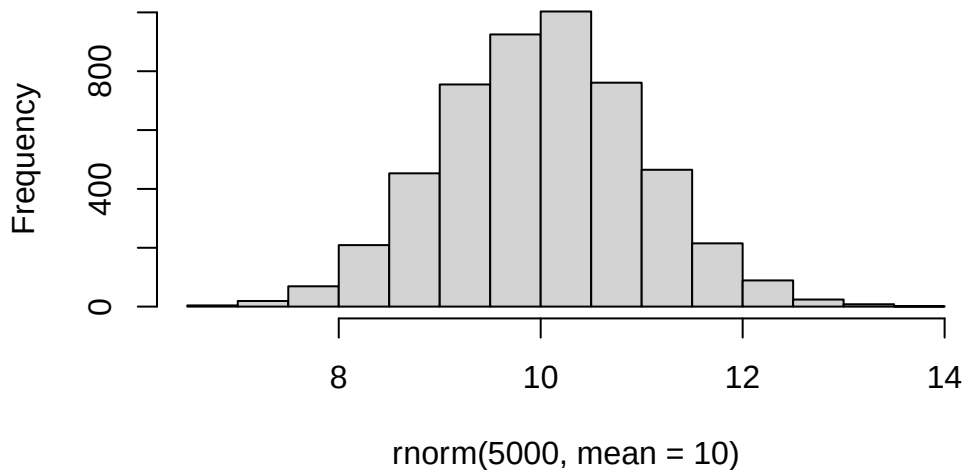
Today we will begin our exploration of some important machine learning methods, namely **clustering** and **dimensionality reduction**.

Let's make up some input data for clustering where we know what the natural "groupings"clusters" are.

The function `rnorm()` can be useful.

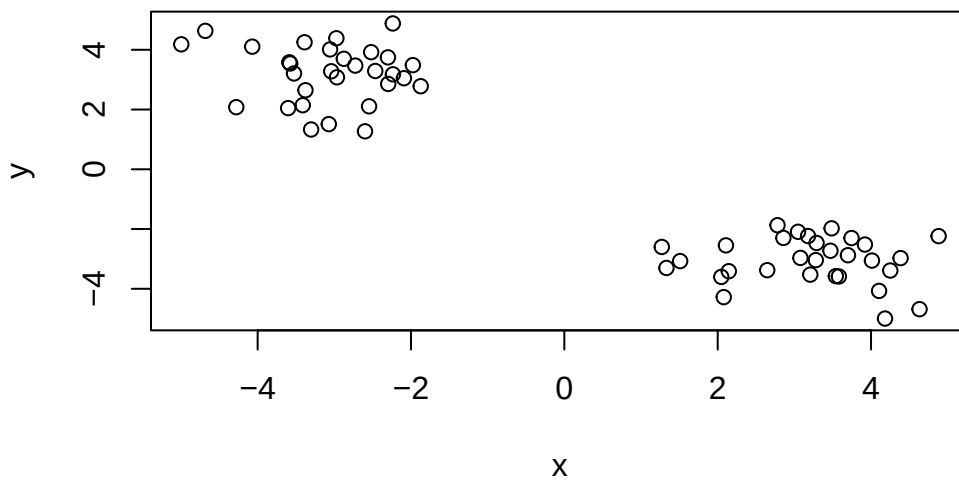
```
hist( rnorm(5000, mean = 10) )
```

Histogram of `rnorm(5000, mean = 10)`



Q. Generate 30 random numbers centered at +3 and 30 random numbers centered at -3

```
tmp <- c(rnorm(30, mean = 3), rnorm(30, mean = -3))  
x <- cbind(x = tmp, y = rev(tmp))  
plot(x)
```



K-means clustering

The main function in “base R” for K-means clustering is called `kmeans()`:

```
km <- kmeans(x, 2)  
km
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	y
1	3.191470	-3.055253

2 -3.055253 3.191470

Clustering vector:

[illegible]

Within cluster sum of squares by cluster:

```
[1] 44.57364 44.57364
      (between_SS / total_SS =  92.9 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

Q. What component of the results object details the cluster sizes?

km\$size

[1] 30 30

Q. What component of the results object details the cluster centers?

km\$centers

	x	y
1	3.191470	-3.055253
2	-3.055253	3.191470

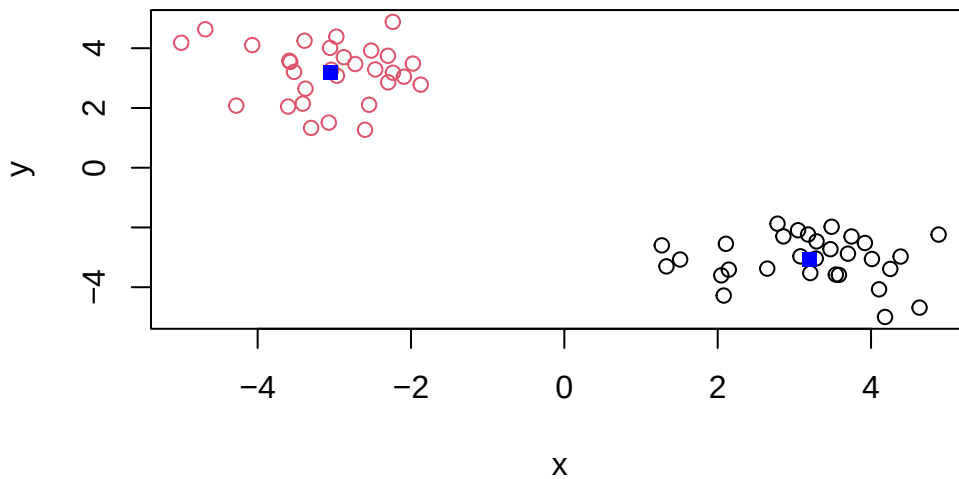
Q. What component of the results object details the cluster membership vector (i.e. our main result of which points lie in which cluster)?

```
km$cluster
```

[illegible]

Q. Plot our clustering results with points colored by cluster and also add the cluster centers as new points colored blue

```
plot(x, col = km$cluster )
points(km$centers, col = "blue", pch = 15)
```



Q. Run `kmeans()` again and this time produce 4 clusters and call your result object `k4`

```
k4 <- kmeans(x, 4)
k4
```

K-means clustering with 4 clusters of sizes 13, 8, 9, 30

Cluster means:

	x	y
1	-2.431505	3.439385
2	-3.273966	1.892019
3	-3.761811	3.988437
4	3.191470	-3.055253

Clustering vector:

```
[1] 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 1 1 1 3 1 2 3 2
[39] 1 2 3 1 2 3 2 1 2 3 1 1 1 1 1 2 3 3 2 3 1 3
```

```

Within cluster sum of squares by cluster:
[1] 5.342540 3.756383 5.517224 44.573640
(between_SS / total_SS = 95.3 %)

```

Available components:

```

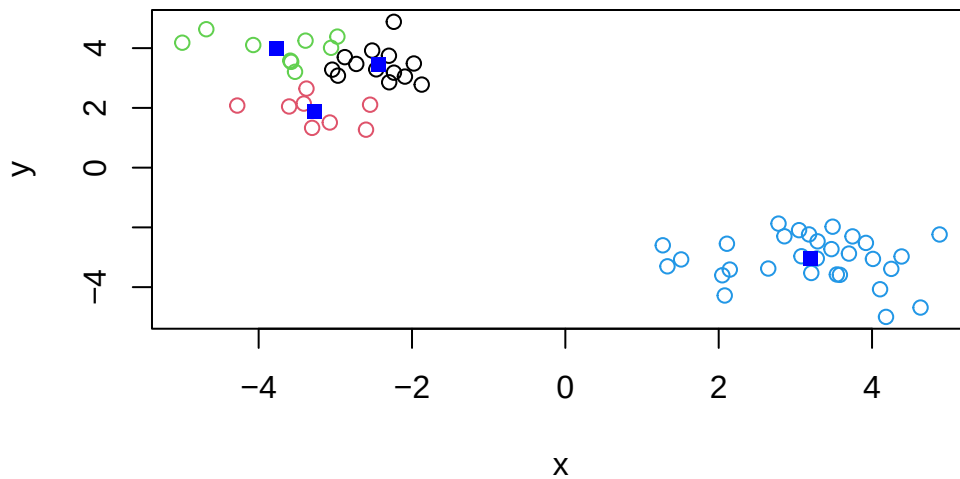
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"

```

```

plot (x, col = k4$cluster)
points(k4$centers, col = "blue", pch = 15)

```



The metric

```
km$tot.withinss
```

```
[1] 89.14728
```

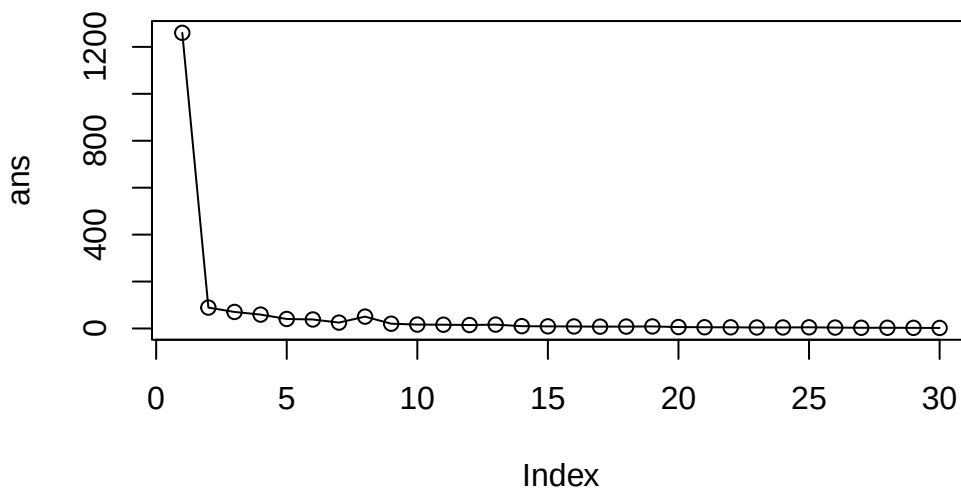
```
k4$tot.withinss
```

```
[1] 59.18979
```

Q. Let's try different number of K (centers) from 1 to 30 and see what the best result is?

```
ans <- NULL
for(i in 1:30) {
  ans <- c(ans, kmeans(x, centers = i)$tot.withinss)
}
```

```
plot(ans, typ = "o")
```



Key-point: K-means will impose a clustering structure on your data even if it is not there - it will always give you the answer you asked for even if that answer is silly!

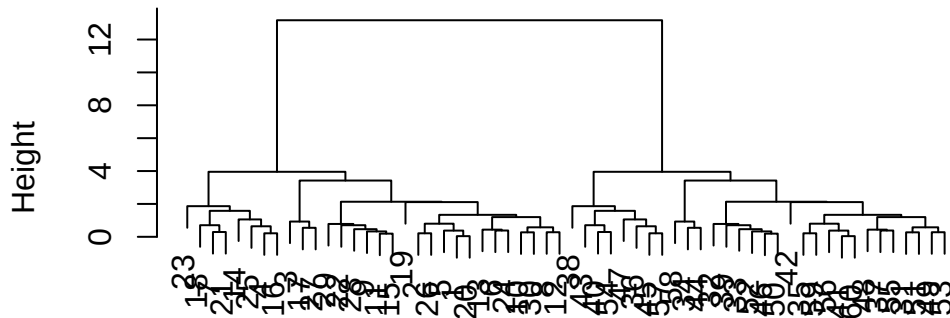
Hierarchical Clustering

The main function for Hierarchical Clustering is called `hclust()`.

Unlike `kmeans()` (which does all the work for you) you can't just pass `hclust()` our raw input data. It needs a "distance matrix" like the one returned from the `dist()` function.

```
d <- dist(x)
hc <- hclust(d)
plot(hc)
```

Cluster Dendrogram

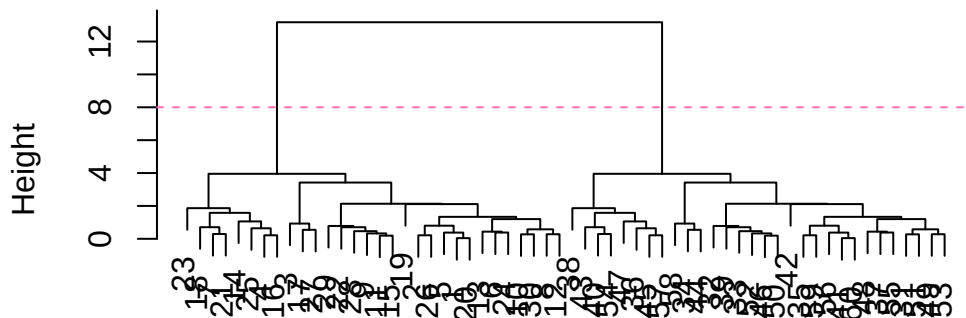


d
hclust (*, "complete")

To extract our cluster membership vector from a `hcluster()` result object we have to “cut” our tree at a given height to yield separate “groups”/“branches”.

```
plot(hc)
abline(h = 8, col= "hotpink", lty = 2)
```

Cluster Dendrogram



```
hclust (*, "complete")
```

To do this we use the `cutree()` function on our `hclust()` object:

```
grps <- cutree(hc, h=8)
grps
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

```
table(grps, km$cluster)
```

```
grps  1  2
    1 30  0
    2  0 30
```

PCA of UK food data

Import the data set on food consumption in the UK

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
x
```


	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139
7	Fresh_potatoes	720	874	566	1033
8	Fresh_Veg	253	265	171	143
9	Other_Veg	488	570	418	355
10	Processed_potatoes	198	203	220	187
11	Processed_Veg	360	365	337	334
12	Fresh_fruit	1102	1137	957	674
13	Cereals	1472	1582	1462	1494
14	Beverages	57	73	53	47
15	Soft_drinks	1374	1256	1572	1506
16	Alcoholic_drinks	375	475	458	135
17	Confectionery	54	64	62	41

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
dim(x)
```

```
[1] 17  5
```

One solution is set the row names is to do it by hand...

```
rownames(x) <- x[,1]
```

To remove the first column I can use the minus index trick

```
x <- x[,-1]
x
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209

Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494
Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

A better way to do this is to set the row names to the first column with `read.csv()`

```
x <- read.csv(url, row.names = 1)
x
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494
Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

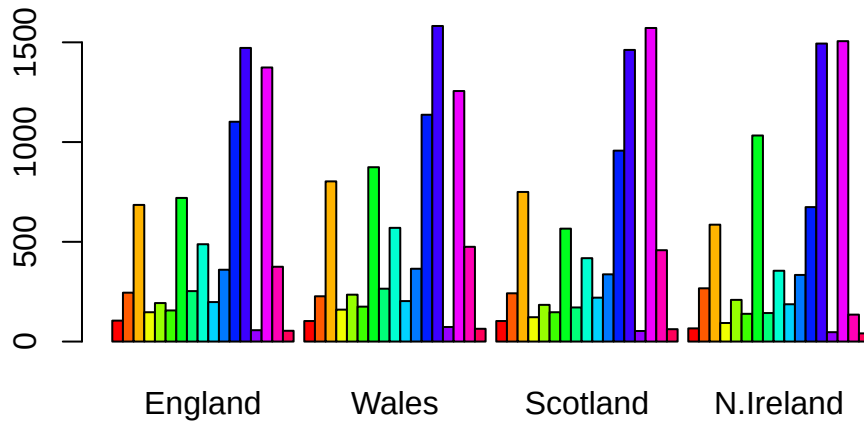
Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer the second way, arguing with `read.csv()` because it’s less work and does it right as I enter the data and not have to do it after!

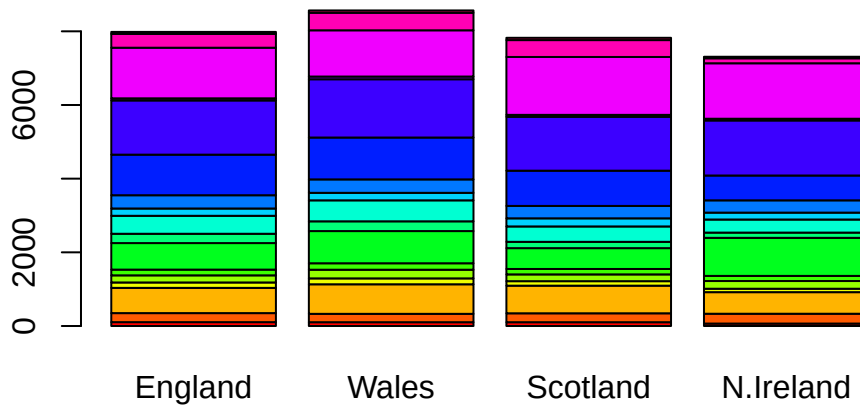
Spotting major differences and trends

Is difficult even in this 17D dataset...

```
barplot(as.matrix(x), beside = T, col = rainbow(nrow(x)))
```



```
barplot(as.matrix(x), beside = F, col = rainbow(nrow(x)))
```



Q3: Changing what optional argument in the above `barplot()` function results in the following plot?

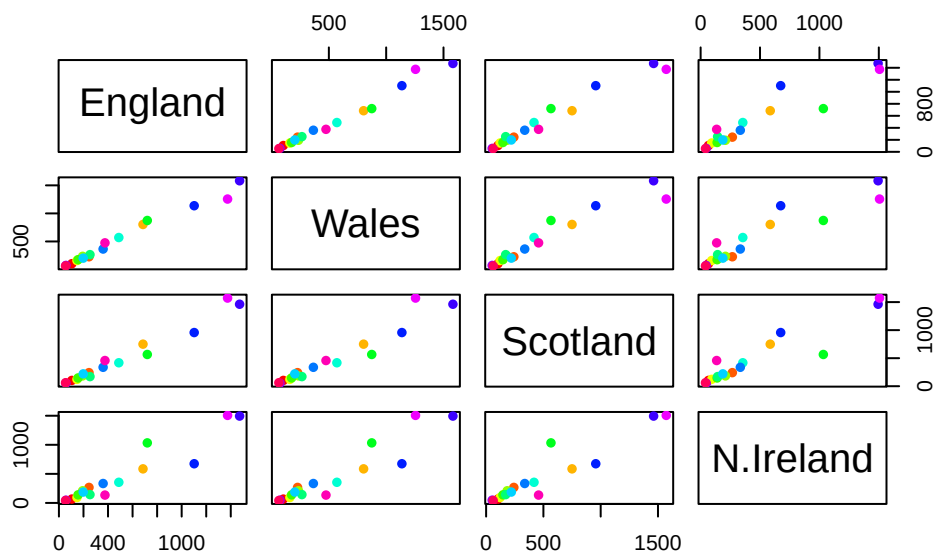
Changing the `beside` = part of the `barplot()` function to `**F*` will give the above plot. The default also makes it this way, so just omitting it would also give this.

Pairs plot and heatmaps

Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

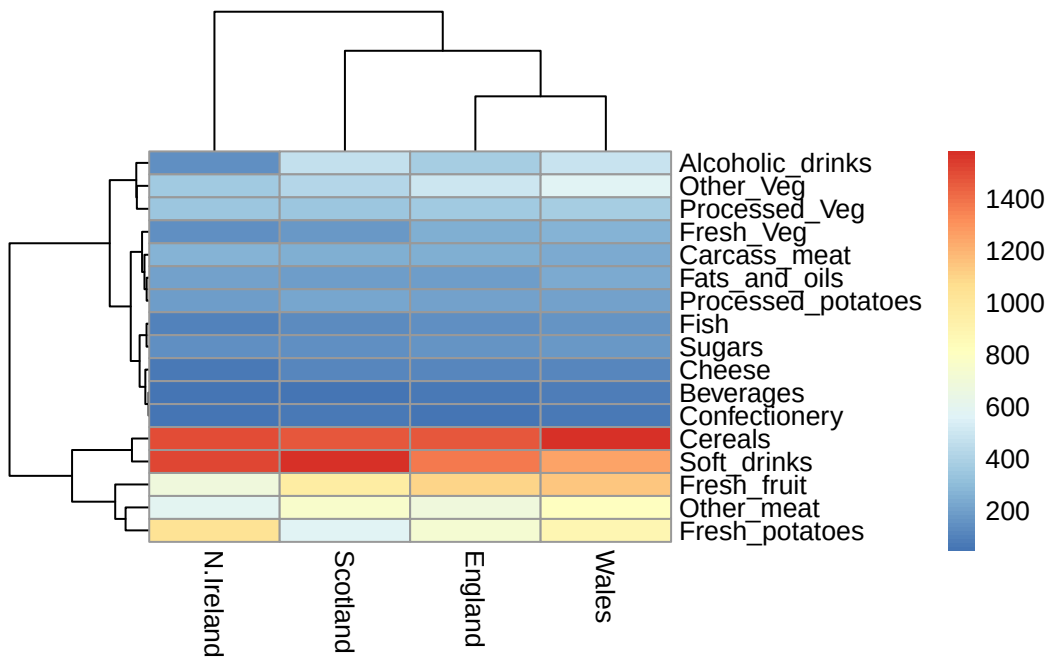
The points on the diagonal are the ones that are similar to the other, if its not within the diagonal they are dissimilar.

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



```
library(pheatmap)

pheatmap( as.matrix(x) )
```



PCA to the rescue

The main PCA function in “base R” is called `prcomp()`. This function wants the transpose of our food data as input (i.e. the foods as columns and the countries as rows).

```
pca <- prcomp(t(x))
```

```
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.7e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.0e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.0e+00

```
attributes(pca)
```

\$names

```
[1] "sdev"      "rotation" "center"    "scale"     "x"
```

\$class

```
[1] "prcomp"
```

To make one of our main PCA result figures we turn to `pca$x` the scores along our new PCs. This is called “PC plot” or “score plot” or “ordination plot”...

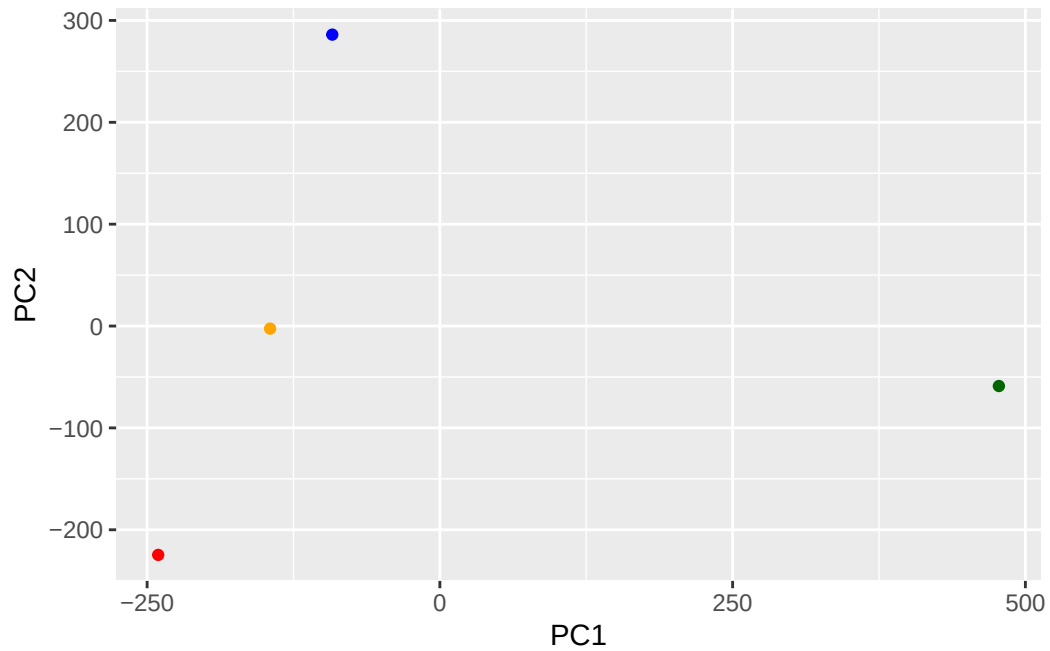
```
pca$x
```

	PC1	PC2	PC3	PC4
England	-144.99315	-2.532999	105.768945	1.612425e-14
Wales	-240.52915	-224.646925	-56.475555	4.751043e-13
Scotland	-91.86934	286.081786	-44.415495	-6.044349e-13
N.Ireland	477.39164	-58.901862	-4.877895	1.145386e-13

```
my_cols <- c("orange", "red", "blue", "darkgreen")
```

```
library(ggplot2)
```

```
ggplot(pca$x) +  
  aes(PC1, PC2) +  
  geom_point(col = my_cols)
```



The second major result figure is called a “loadings plot” or “variable contributions plot” or “weight lot”

```
ggplot(pca$rotation) +  
  aes(PC1, rownames(pca$rotation)) +  
  geom_col()
```

